HSLU Hochschule Luzern

Chat with Your Data:

A Hands-On Introduction to Retrieval-Augmented Generation (RAG)

Dr. Aygul Zagidullina and Dr. Elena Nazarenko

Informatik
*

FH Zentralschweiz

Agenda

1. **Introduction to RAG**
2. **Foundation Models and Architecture**
   - Building Blocks of RAG
   - RAG Architecture and Integration
   - Implementation Steps

3. **Tools for RAG**
   - LlamaIndex
   - LangChain
   - OpenAI Assistants (GPTs)

4. **LangChain Components**
   - Chains
   - LLMChain
   - LangGraph
5. **Agent concepts**

01
Introduction to RAG

Understanding Retrieval-Augmented Generation

Fundamentals and Core Concepts

🤖 RAG Fun Facts

1. **RAG was pioneered by Meta AI in 2020**
Researchers introduced RAG as a way to combine retrieval and generation, enhancing factual accuracy.
https://arxiv.org/abs/2005.11401

2. **RAG reduces hallucinations—but doesn’t eliminate them!**
Even with external retrieval, AI can still make things up if the right data isn’t available.

3. **Early search engines pioneered retrieval techniques!**
Before RAG, search engines like AltaVista (1995) and Google (1998) developed sophisticated information retrieval methods, including TF-IDF and PageRank, laying the groundwork for modern neural retrieval systems.

4. **"Chunking" can make or break a RAG system.**
If documents are split incorrectly, the model may retrieve unrelated or misleading information. Proper chunking is key!

5. **RAG mimics human cognition!**
Cognitive scientists have found that humans don’t just memorize facts—we retrieve memories and generate new ideas by combining past knowledge creatively. RAG does the same!

Foundation Models: The Building Blocks of RAG

1 Vast Knowledge Base

Foundation models, trained on massive text datasets, possess an extensive knowledge base encompassing various domains and topics. This knowledge is encoded within the model's parameters, allowing it to generate coherent and informative responses.

2 General Purpose

Foundation models are designed for general-purpose tasks, such as text generation, translation, and summarization. However, their broad knowledge base may not always encompass the nuances of specialized domains.

3 Domain Specialization

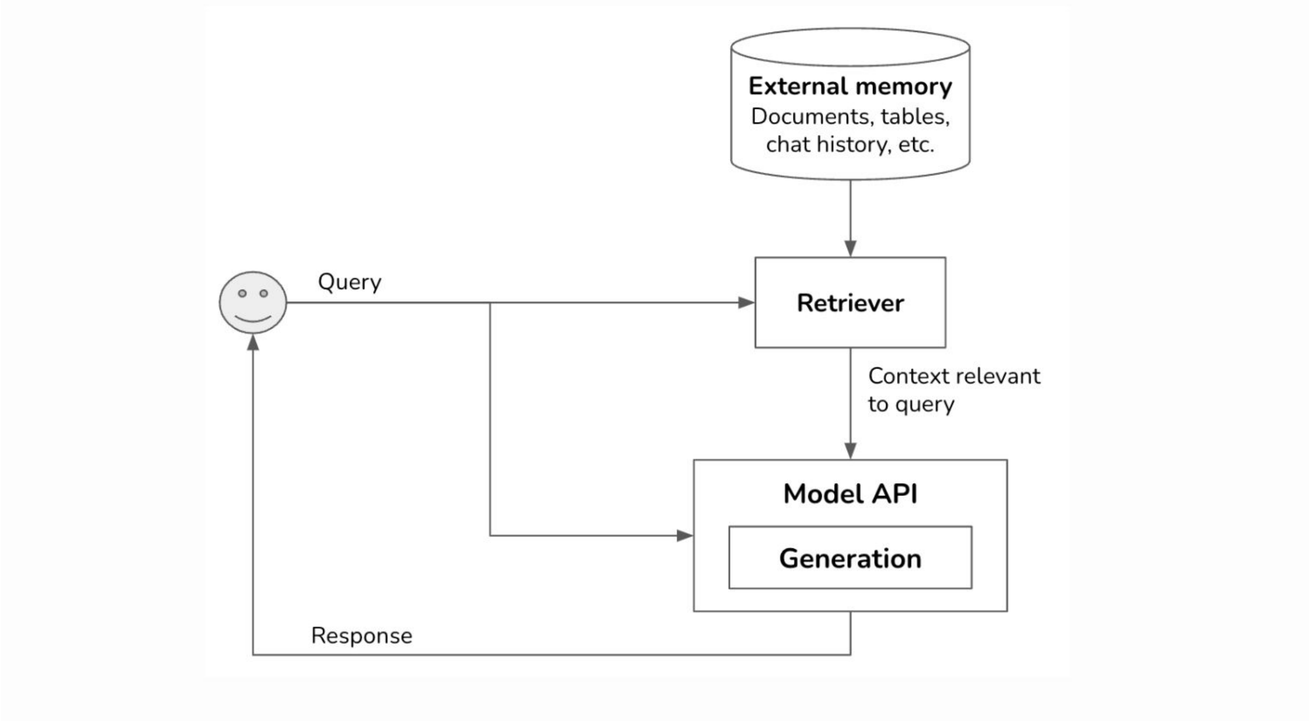
While foundation models offer a robust starting point, domain-specific knowledge is often crucial for achieving high-quality results in specialized tasks. This is where retrieval augmented generation comes into play.

02
Foundation Models and Architecture

- Building Blocks of RAG
- RAG Architecture and Integration
- Implementation Steps

Retrieval-Augmented Generation

The most well-known pattern for context construction is RAG, Retrieval-Augmented Generation. RAG consists of two components: a generator (e.g. a language model) and a retriever, which retrieves relevant information from external sources.



Note: Retrieval isn't unique to RAGs. It's the backbone of search engines, recommender systems, log analytics, etc. Many retrieval algorithms developed for traditional retrieval systems can be used for RAGs.

RAG Architecture: Integrating Retrieval and Generation

1

Input Request

The process begins with a user request or input query. This query can be a question, a statement, or a task that requires domain-specific knowledge.

2

Retrieval

The RAG system utilizes an information retrieval component to fetch relevant documents or data points from the designated knowledge base.

3

Context Enrichment

The retrieved information is then used to enrich the context of the input request. This context augmentation helps the foundation model understand the nuances of the domain and generate more accurate responses.

4

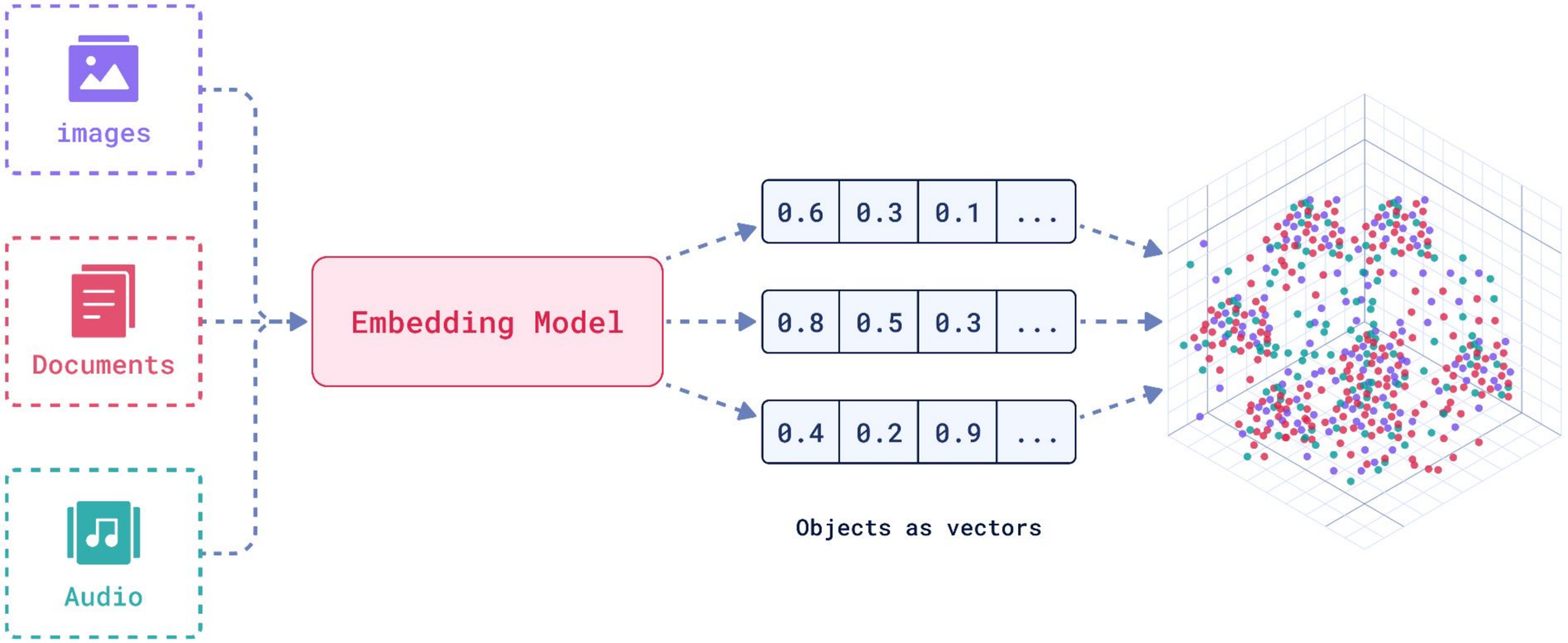
Generation

Finally, the foundation model uses the enriched context to generate a response to the input request. This response leverages both the general knowledge of the foundation model and the specific information retrieved from the knowledge base.

Applications of RAG: Enhancing Domain-Specific Tasks

Domain	Task	Example
Healthcare	Medical diagnosis	A RAG system can be used to analyze patient symptoms and medical records, providing insights to assist doctors in making diagnoses.
Finance	Financial analysis	RAG systems can be used to analyze financial reports and market data, providing insights to help investors make informed decisions.
Law	Legal research	RAG systems can be used to search legal databases and retrieve relevant precedents, assisting lawyers in building their cases.

How does it work? You need an embedding model



Vector embeddings convert text into numbers that capture meaning. Similar texts get similar numbers, making it possible to find related information quickly.

Stages within RAG

Loading: Retrieve data from text files, PDFs, databases, APIs, or websites.

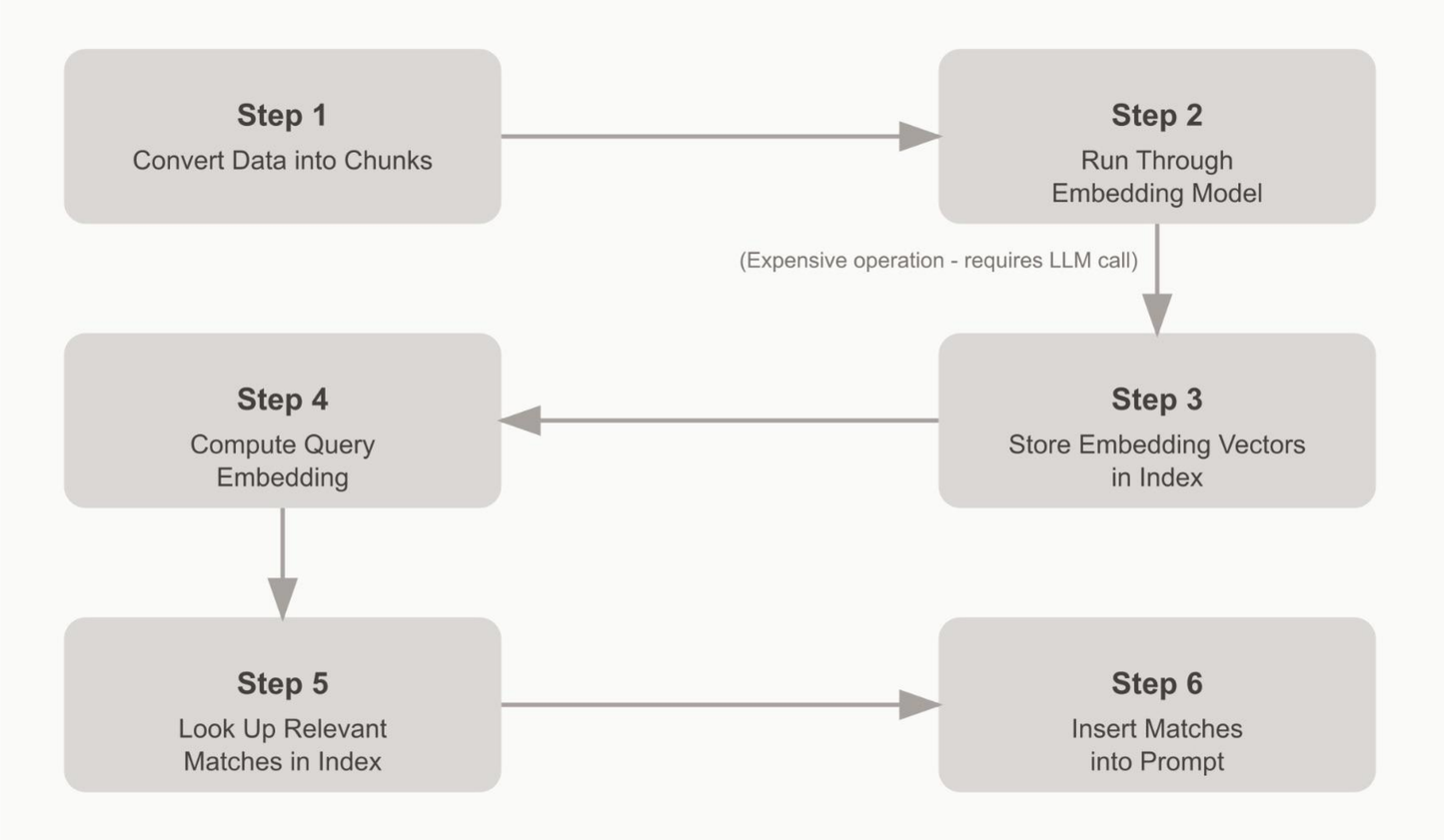
Indexing:
- Create a searchable data structure for querying.
- Generate vector embeddings (numerical representations of meaning).
- Use metadata strategies to improve relevance.

Storing: Save indexed data and metadata to avoid re-processing.

Querying:
- Use sub-queries, multi-step queries, or hybrid search methods.
- Leverage LLMs + LlamaIndex for better responses.

Evaluation:
- Assess accuracy, faithfulness, and speed of responses.
- Compare different strategies for optimization.

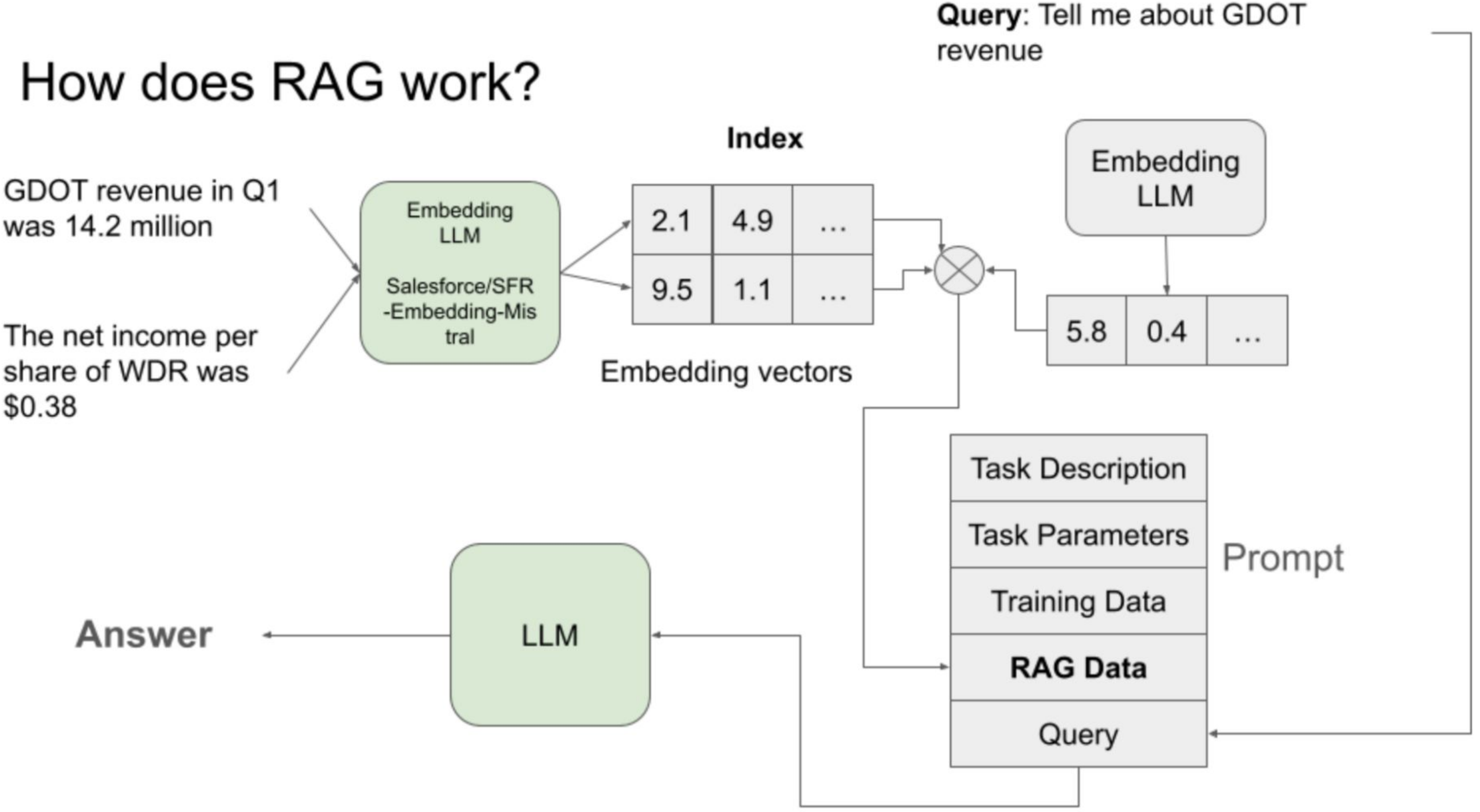
RAG Implementation Steps



Important Considerations

- Chunking affects context relevance and retrieval
- Vector quality depends on embedding model selection
- Retrieval accuracy impacts response quality
- Overall performance depends on the efficiency of both retrieval and LLM processing

How does RAG work?



03
Tools for RAG

- LlamaIndex
- LangChain
- OpenAI Assistants (GPTs)

Overview of Tools for RAG

LlamaIndex:

https://www.llamaindex.ai/

Framework for building LLM applications
Robust data handling and retrieval capabilities
Perfect for data-centric applications like RAG
Efficient indexing and retrieval methods
Better chunking strategies
Multimodality
Free

LangChain:

https://www.langchain.com

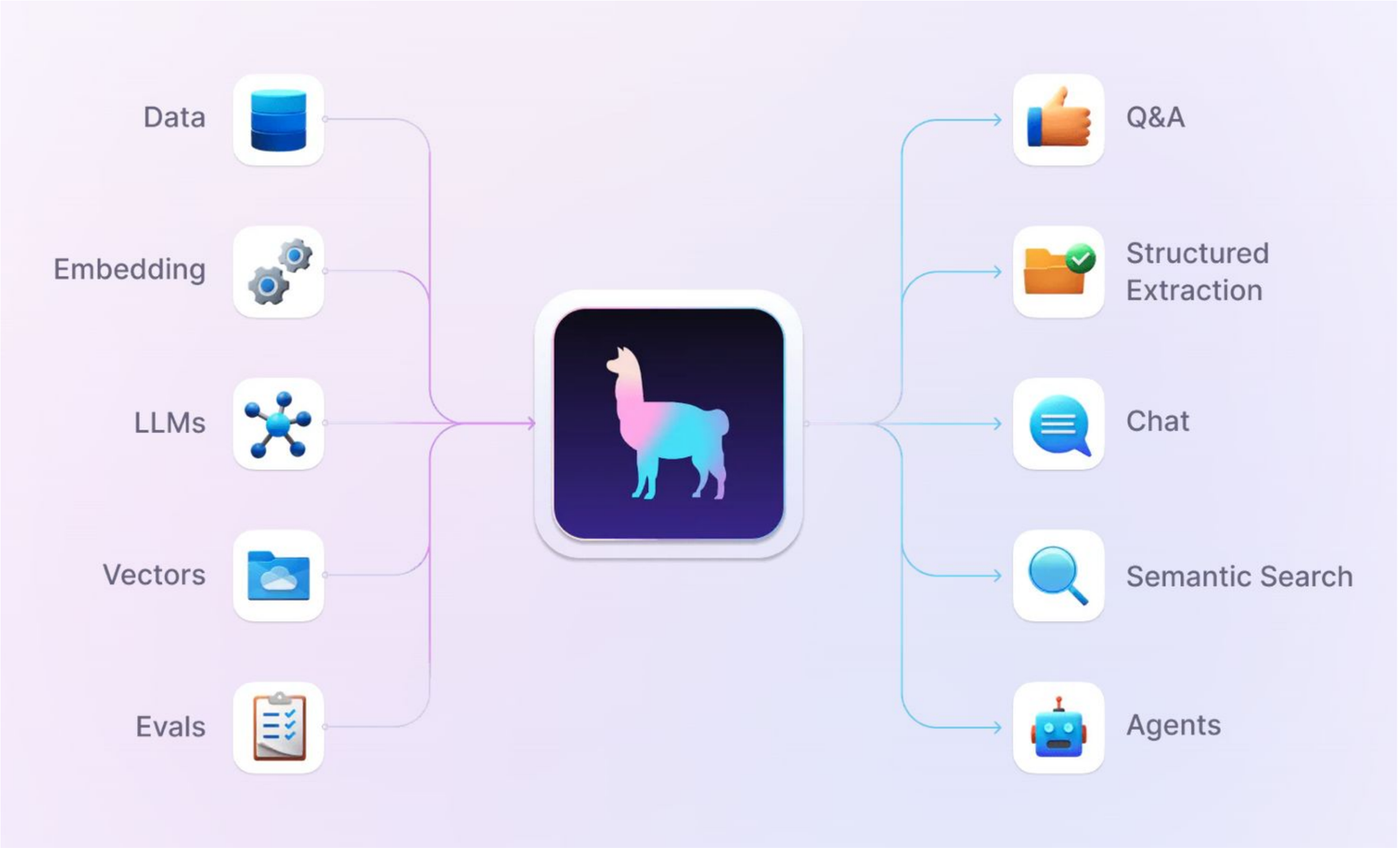
Provides a balance of customization and ease of use
Ideal for developers seeking flexibility in their LLM interactions
Free

OpenAI Assistants (GPTs):

https://platform.openai.com/docs/assistants/overview

Accessible and quick-to-deploy option
Suitable to integrate LLMs without deep technical expertise
To create quick proofs of concepts
Paid

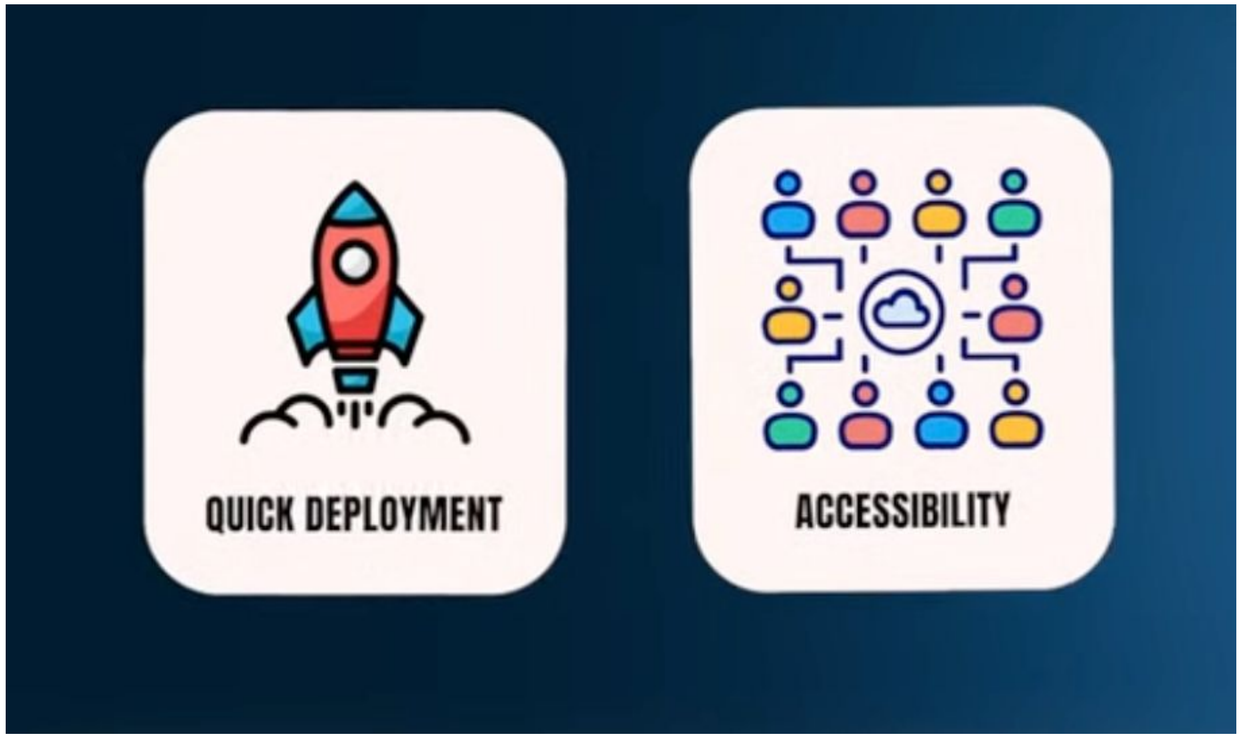
LlamaIndex



- Go-to for a RAG-based application
- Offers Fine-Tuning and Embedding optimizations
- Free
- Open-Source
- Continually Developed

https://www.llamaindex.ai/

OpenAI GPTs & assistants



Pro:

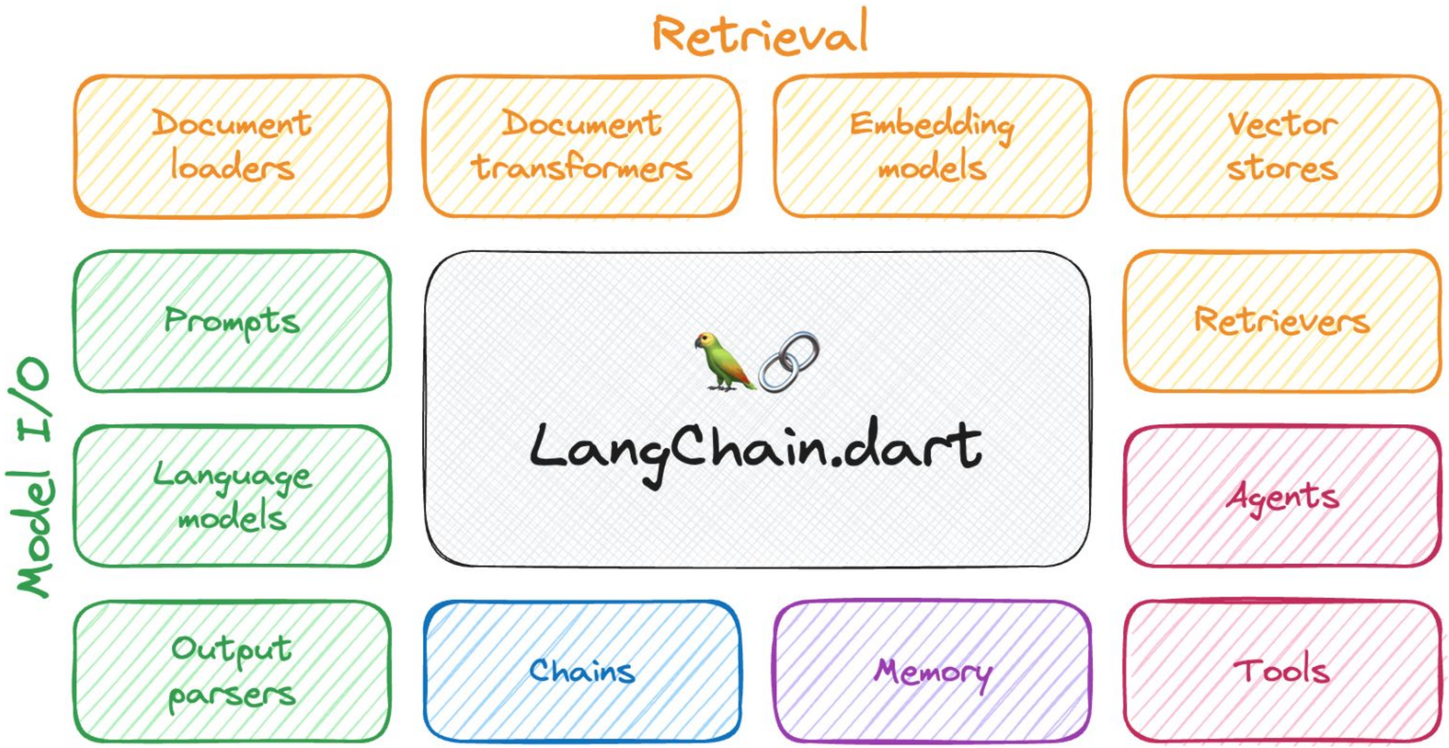
Streamlined experience
User-friendly interface

Cons:

Dependent on OpenAI (problems with data privacy and safety)
Not enough unique value

https://platform.openai.com/docs/assistants/overview

LangChain



Source

Powerful and flexible framework for building applications with LLMs

- Stands out for its ability to integrate seamlessly with various LLM providers like OpenAI, Cohere, HuggingFace.
- Mostly used to create applications that can process user input text and retrieve relevant responses leveraging the latest NLP.

Key advantages:

- Support for prompt engineering (provides prompt templates).
- Structured output from LLMs (for example, JSON objects).
- Nice middle ground for a balance between customization and ease of use.

https://www.langchain.com/

Chat with Data: Retrieval Augmented Generation (RAG) with LlamaIndex and OpenAI API

Make a copy (File → Save a copy in Drive) to modify it

04
LangChain Components

- Chains
- LLMChain
- LangGraph

LangChain - Chains

Chains

A sequence of calls that you can make to an LLM, and they let you go beyond a single API call. You can chain them together to make multiple calls to be executed in a specific order.

Core Concept in LangChain:
Enables logical sequencing beyond single API calls.

Purpose of Chains:

- Break down complex tasks into sequential steps.
- Leverage strengths of different models/utilities.
- Maintain state and memory between calls.
- Enable processing, filtering, and validation.
- Facilitate debugging and instrumentation.

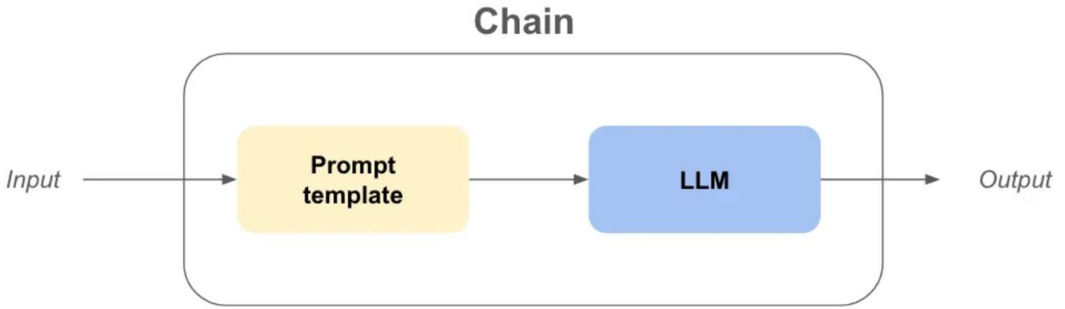
LangChain - 🦜 LLMChain

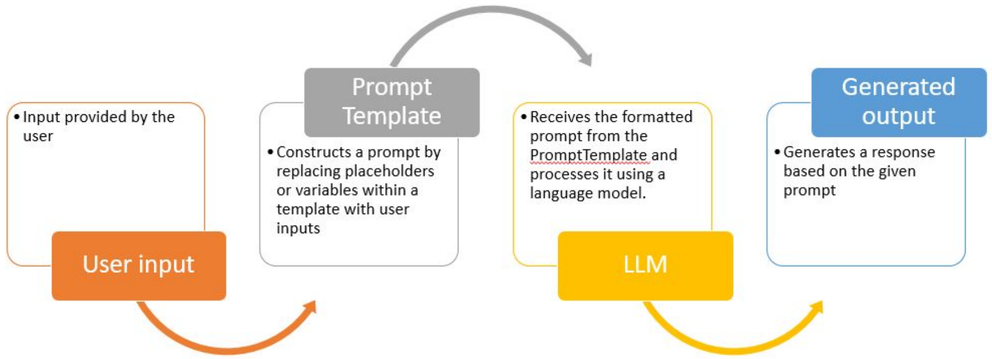
🦜 LLMChain is the most commonly used type of chain.

The LLMChain consists of a PromptTemplate, a language model, and an optional output parser.

Advantages over direct LLM prompts:

- Breaks complex prompts into simpler, sequential ones.
- Maintains state and memory between prompts.
- Enables preprocessing, validation, and debugging.



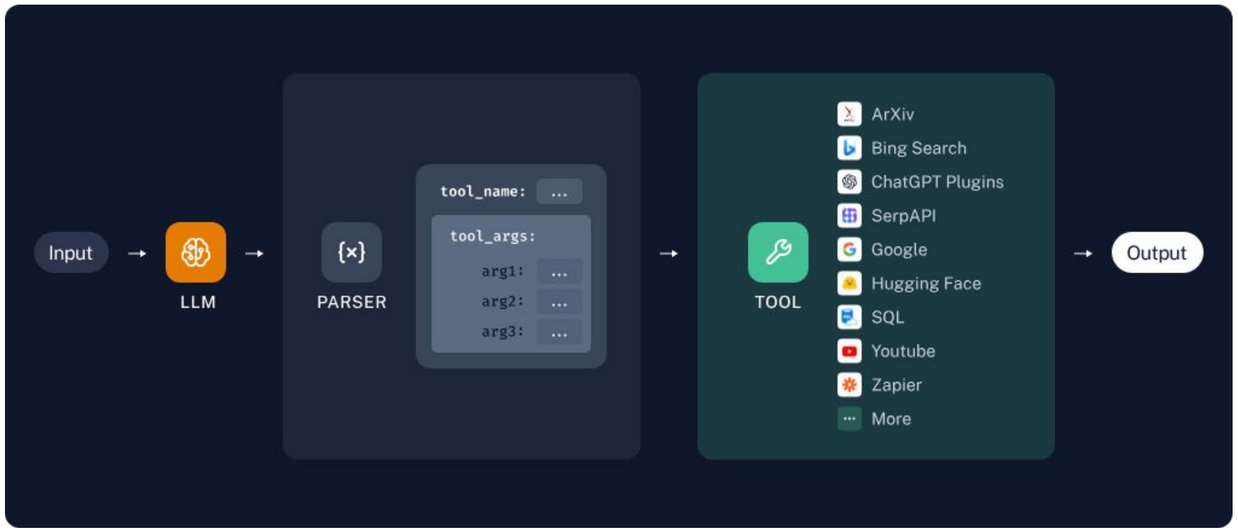


LangChain - 🦜 LLMChain

```
from langchain import PromptTemplate, OpenAI, LLMChain

# the language model
llm = OpenAI(temperature=0)

# the prompt template
prompt_template = "Act like a comedian and write a super funny two-sentence
short story about {thing}?"

llm_chain = LLMChain(
    llm=llm,
    prompt=PromptTemplate.from_template(prompt_template)
)

llm_chain("A toddler hiding his dad's laptop")
```

LangGraph 🦜🕸

TL;DR: LangGraph is module built on top of LangChain to better enable creation of cyclical graphs, often needed for agent runtimes.



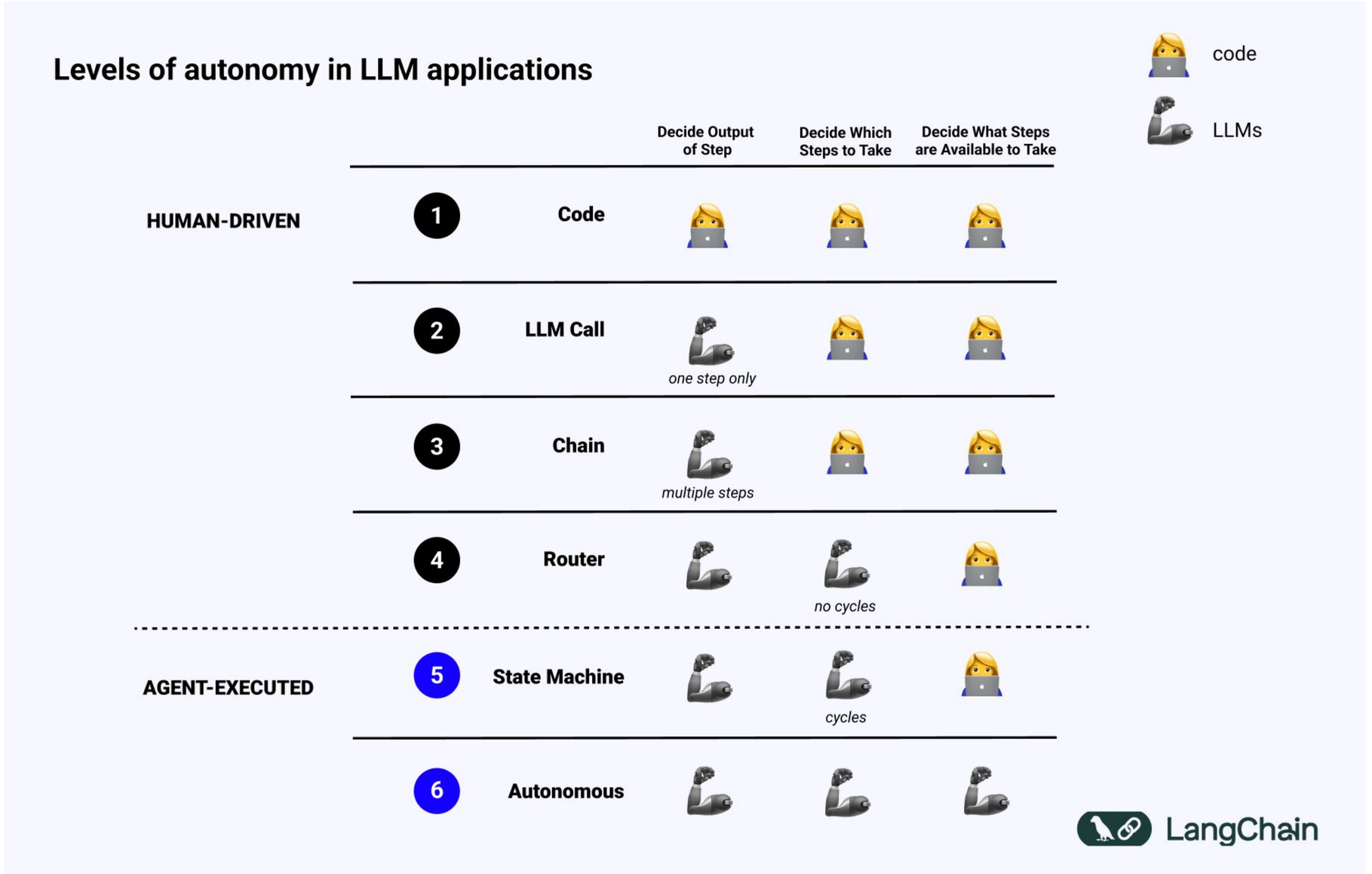
05
Agent Concepts

Understanding AI Agents in RAG Systems

Advanced Applications and Use Cases

Agents

> 💡 An AI agent is a system that uses an LLM to decide the control flow of an application.



Build a Question-Answering System over SQL data

Make a copy (File → Save a copy in Drive) to modify it